Backend Schema (Proposed)

05 — Backend Schema (Proposed)

*Last updated: 2026-06-08 — **PLANNING ONLY. No backend is built yet.***

> The frontend was deliberately kept backend-agnostic. This document proposes a data model and API so that whoever builds the backend (and whichever technology is chosen) starts from a shared plan. **Nothing here is implemented.** It will be updated/replaced once the backend technology is chosen.

1. Decision still open

The backend technology is **undecided** (Supabase was removed from the shell). The schema below is written in neutral relational terms; it maps cleanly to either Postgres (custom Node/Express/Nest or Supabase) or another relational store.

2. Guiding principles

- **One user identity** across the whole suite (single sign-in).
- **Per-tool data tables**, namespaced, mirroring today's `localStorage` namespaces — so migration is a per-tool lift-and-shift.
- **Offline-friendly:** the client keeps the localStorage path as a fallback; the API is the source of truth when configured (mirror the existing `isConfigured` auth pattern).

3. Core identity

```
users
  id            uuid  PK
  email         text  unique
  name          text
  role          text  -- 'admin' | 'consultant' | 'client'
  status        text  -- 'Active' | 'Pending'
  created_at    timestamptz
```

4. Per-tool tables (proposed)

Power Planner

```
pp_weeks            id, user_id FK, week_start date, data jsonb, updated_at
pp_schedule         id, user_id FK, schedule jsonb           -- variable-length weeks
pp_settings         id, user_id FK, custom_options jsonb, start_date date
-- mirrors keys: power-planner-weekly-data-v5, -schedule-v1, -start-date-v1, -custom-options-v1
```

Reasons Eliminator

```
re_sessions         id, user_id FK, payload jsonb, created_at
re_grip_tests       id, user_id FK, payload jsonb, created_at
re_grip_history     id, user_id FK, payload jsonb, created_at
-- mirrors keys: altus.reasonEliminator.sessions.v1, .gripTest.v1, .gripHistory.v1
```

Meeting Success Maximizer

```
msm_meetings        id, user_id FK, meeting jsonb, created_at, updated_at
-- mirrors key: msm_meetings  (msm_sidebar_collapsed stays a local UI pref)
```

Time Finder

```
tf_assessments      id, user_id FK, assessment jsonb, archived bool, created_at
-- mirrors keys: assessments, currentAssessment, editAssessment, archivedAssessments
-- NOTE: namespace these client keys (timefinder.*) when wiring the API.
```

Time Auditor (shell)

```
ta_entries          id, user_id FK, entry jsonb, created_at
-- mirrors key: ta_saved_entries
```

> Most tools currently store rich nested objects, hence `jsonb` blobs as a first migration step. These can be normalized into proper columns/relations later, per tool, once the live shapes are confirmed.

5. Proposed API surface (REST, illustrative)

```
POST   /api/auth/login            → { user, token }
POST   /api/auth/register
GET    /api/me

GET    /api/power-planner/weeks
PUT    /api/power-planner/weeks/:weekStart
GET    /api/reasons/sessions
POST   /api/reasons/sessions
GET    /api/meetings
POST   /api/meetings
PUT    /api/meetings/:id
GET    /api/time-finder/assessments
POST   /api/time-finder/assessments
GET    /api/time-auditor/entries
POST   /api/time-auditor/entries
```

Auth via bearer token; every tool route scoped to the authenticated `user_id`.

6. Integration plan (when backend starts)

1. Pick the backend tech; scaffold under `../backend/`.
2. Add an API base URL + `isConfigured`-style flag to the frontend env.
3. Per tool, introduce a thin **data layer** that reads/writes the API and falls back to `localStorage` — one tool at a time, lowest-risk first (e.g. Meeting).
4. Migrate existing local data on first authenticated load (optional import step).
5. Update Architecture §5 and App Flow §4 as each tool goes live on the API.
